

Bruce Smoochie: Wireless Hacking Device

Group 5 Final Report

Dhameliya Arpit
Capstone Class
Pace University
New York USA
ad18846n@pace.edu

Zeel Patel
Capstone Class
Pace University
New York USA
zp67345n@pace.edu

Harsh Vithani
Capstone Class
Pace University
New York USA
hv39353n@pace.edu

Abstract—The Smoochie Board is a portable wireless security testing device built around an ESP32-S3 microcontroller and the open-source Bruce firmware, designed to bring several wireless technologies together in one compact tool. It includes a 2.4 GHz radio module, a sub-GHz radio module, an infrared receiver and transmitter, a GPS unit for location tagging, and an RFID/NFC reader for scanning access cards, all supported by internal flash storage that removes the need for an SD card. With a built-in lithium-ion battery and charging circuit, the board now operates fully on its own as a handheld device. It can scan nearby wireless activity, record and replay simple signals, read RFID tags, capture and replay infrared signals, and log observations with GPS context. These features make it useful for legal penetration testing, red-team learning, and helping students understand how wireless systems work and where weaknesses often appear. The goal is to provide a simple, affordable way to explore real-world wireless behavior without needing multiple separate tools. By keeping the design lightweight and easy to operate, the Smoochie Board makes wireless security testing more accessible to beginners while still offering enough capability for more advanced users. This report summarizes the project's purpose, relevant background, the development approach, the results achieved so far, and how the final device meets the goals set for Progress Report 4.

Keywords—wireless security, IoT security, ESP32, penetration testing, radio-frequency analysis, Bruce firmware, Smoochie board

I. INTRODUCTION

Wireless devices are deeply integrated into homes, businesses, and industrial environments, making wireless protocol weaknesses a meaningful attack surface for adversaries. Organizations continue adopting inexpensive IoT technologies, yet many consumer devices rely on unencrypted or proprietary communication methods. This creates opportunities for replay attacks, signal injections, location tracking, and device manipulation. The goal of this project is to design and implement the Smoochie Board, a compact, multi-radio wireless security testing platform intended for education, penetration testing, and red-team demonstrations. The system aims to unify several commonly exploited wireless communication technologies, including 2.4 GHz proprietary radios, sub-GHz radios, infrared signaling, and geolocation, into a single portable device running enhanced Bruce firmware. The scope of this project includes ESP32-S3 hardware integration, multi-radio firmware development, GPS-based event tagging, IR signal capture, and a user-friendly menu interface supporting reconnaissance and replay workflows. The project does not attempt to crack encrypted Wi-Fi protocols or perform non-permitted jamming; instead, it focuses on legal demonstration of inherent weaknesses in unencrypted or poorly implemented wireless systems.

The research problem addressed in this work is the lack of an affordable, extensible, open-source tool that consolidates multiple RF and IR communication channels into one device. Existing options such as SDR platforms require advanced RF knowledge, while other tools focus on only one protocol at a time. The Smoochie Board contributes by offering a customizable development platform that simplifies hands-on exploration of RF vulnerabilities and supports cybersecurity education. This also helps bridge the skill gap for beginners who often struggle to access practical RF tools without high upfront costs. By lowering the entry barrier, the platform encourages more experimentation and safer learning in controlled environments. Ultimately, the project provides a pathway for students and practitioners to understand real wireless weaknesses before encountering them in the field. The device also creates opportunities for instructors to demonstrate common attack concepts in a predictable and repeatable way. In professional environments, it offers a lightweight option for teams that need quick reconnaissance without deploying complex RF setups. Taken together, these benefits underscore the broader educational and practical value of developing an accessible multi-radio testing platform.

A. *Project Goal*

The primary goal of this project is to design and implement the Smoochie Board, a compact multi-radio wireless security testing platform built on an ESP32-S3 microcontroller and the Bruce firmware. The device aims to act as a single, portable tool that can:

- Scan and monitor multiple wireless bands
- Capture and replay simple RF and IR signals
- Read and experiment with basic RFID/NFC tags
- Associate wireless observations with GPS coordinates
- Operate as a self-contained unit with a battery and internal flash storage

The board is intended for *authorized* security testing, lab exercises, and red-team style demonstrations in controlled environments, so the project intentionally avoids features that target encrypted Wi-Fi cracking or disruptive jamming beyond narrow, ethical test scenarios.

B. Scope of the Project

The scope of this project covers both hardware and firmware aspects:

1. **Hardware integration** of the ESP32-S3 core, nRF24L01, CC1101, PN532, IR transceiver, GPS module, TFT display, navigation buttons, lithium-ion battery, and TP4056 charging circuit into a compact, handheld layout.
2. **Firmware integration** using Bruce firmware as the base, including SPI and I2C driver configuration, pin assignments for all modules, interrupt handling, menu navigation, and basic capture and replay flows.
3. **Storage and configuration** using the ESP32-S3's internal flash with LittleFS. This is used for persistent configuration, small logs, replay buffers, and simple data structures instead of an external SD card.
4. **Task scheduling and project management**, including defining specific tasks with owners and deadlines, tracking completion status, and aligning work with the course milestones.
5. **Evaluation of preliminary results**, such as module-level testing, device stability under battery power, and initial replay and RFID demonstrations.

The project does **not** include:

- Design or use of high-power transmit chains meant for long-range jamming
- Attempting to break strong cryptographic schemes
- Advanced SDR-level analysis of arbitrary waveforms

Instead, the focus is on **practical, accessible wireless security testing** that aligns with what a typical penetration tester or student might encounter with low-cost IoT devices.

II. LITERATURE REVIEW

Research in wireless security continues to highlight how consumer IoT devices use numerous non-Wi-Fi protocols that lack encryption, authentication, or replay protection. These gaps create a broad attack surface for adversaries. The following review synthesizes findings from wireless security literature relevant to the design of the Smoochie Board.

A. *Evolution of Wireless Security Assessment Tools*

Early wireless testing tools focused primarily on Wi-Fi networks and used capture adapters and frameworks such as Aircrack-ng and Kismet [1]. These were effective for 802.11 but ignored non-Wi-Fi IoT channels such as 315 MHz, 433 MHz, and infrared. As IoT devices became widespread, researchers emphasized that consumer devices like alarms, HVAC systems, baby monitors, and smart remotes often rely on proprietary sub-GHz or IR protocols with minimal security controls [2].

To analyze these protocols, researchers increasingly rely on software-defined radios (SDRs) such as RTL-SDR and HackRF One [3]. SDRs allow advanced analysis across broad frequency ranges, but they require significant RF expertise, driver configuration, and signal processing skills. This makes them less practical for introductory classroom environments or portable field assessments. As a result, researchers identified the need for compact, dedicated, multi-radio testbeds.

B. *Multi-Frequency IoT Vulnerabilities*

Many IoT devices communicate on unlicensed spectrum including 315 MHz, 433 MHz, 868 MHz, 915 MHz, and 2.4 GHz. Studies document common issues such as fixed codes, replayable packets, lack of rolling keys, ASK/OOK modulation weaknesses, and inconsistent authentication [4]. Work by Shrestha et al. shows that sub-GHz doorbells and smart gate openers leak identifiers and allow replay because of weak protocol design [5]. Similarly, proprietary 2.4 GHz sensor devices have been found to transmit unencrypted telemetry that can be captured using simple transceivers like the nRF24L01 [6].

- **Sub-GHz & 2.4 GHz:** Studies by Shrestha et al. [5] demonstrate that smart gate openers and doorbells frequently use fixed codes or weak rolling codes that are susceptible to "RollJam" or simple replay attacks. Similarly, proprietary 2.4 GHz peripherals (mice, keyboards) often transmit keystrokes in cleartext, which can be intercepted by low-cost transceivers like the nRF24L01 [6].
- **Infrared (IR):** While often considered legacy technology, IR remains dominant in HVAC and AV control. Literature indicates that these protocols (NEC, RC5) lack any form of authentication, making them trivial to capture and replay for unauthorized control [7].

C. *Embedded Platforms for Wireless Testing*

Recent research explores the use of microcontrollers and embedded devices as specialized wireless testing platforms. ESP32 has become popular due to its integrated Wi-Fi and Bluetooth radios, dual-core processing capability, and low cost. Studies on embedded Wi-Fi attack tools and BLE analysis platforms show that the ESP32 can efficiently run real-time scanning, packet injection, and device fingerprinting routines.

Projects like Flipper Zero, MouseJack, and the Yard Stick One demonstrate the practicality of having multiple radios in a single handheld device. However, many of these platforms rely on proprietary firmware or closed development ecosystems. The academic community notes the importance of open-source frameworks that can be modified for custom boards, especially for controlled educational environments. This aligns with the philosophy behind Bruce firmware, which consolidates several radio interfaces while remaining adaptable to custom hardware pinouts and module combinations.

D. Bruce Firmware as a Modular RF Research Framework

Bruce firmware provides an integrated interface for Wi-Fi scanning, BLE enumeration, sub-GHz signaling, infrared operations, and RFID interactions [1]. The modularity of Bruce allows researchers and practitioners to extend the firmware by modifying pin assignments and enabling only the modules relevant to specific hardware. Several community contributions highlight its ability to coordinate cross-radio actions, log events with timestamps, and unify multiple testing actions under a single user interface.

Compared to multi-tool workflows typically required in wireless testing, Bruce reduces operational complexity. Instead of switching between SDR software, IR capture tools, and proprietary RF dongles, the user interacts through a consistent menu system. This allows students and red team operators to execute controlled demonstrations without extensive setup overhead. Because Bruce is open source, it also serves as a development platform for custom academic experiments focused on wireless protocol weaknesses.

E. Importance of Integrated Geolocation and Logging

Literature on RF reconnaissance emphasizes the value of geotagged data for mapping wireless coverage, correlating vulnerabilities with physical locations, and documenting findings for security reports. GPS-assisted scanning has been used extensively in wardriving studies, drone-based RF surveys, and outdoor IoT deployments. Geolocation improves traceability by linking captured signals and device identifiers to specific rooms, floors, or outdoor areas. This practice enhances actionable remediation planning, as physical context often determines feasible countermeasures. By integrating GPS into the Smoochie platform, this project aligns with best practices recommended in wireless audit research: precise location tagging, timestamping, and the ability to reconstruct events after field activities. Such functionality also supports training exercises where students examine the relationship between RF propagation and physical structures.

F. RFID and NFC Security Risks

With the addition of the PN532 module, understanding RFID vulnerabilities is critical.

- **Cloning and Emulation:** The PN532 chipset is a standard tool in security research for interacting with 13.56 MHz tags (ISO/IEC 14443). Research indicates that legacy cards, particularly the MIFARE Classic series, rely on the proprietary Crypto-1 algorithm, which has been cryptographically broken. Tools utilizing the PN532 can perform "nested" authentication attacks to recover keys and clone cards [13], [14].
- **Replay Attacks:** Even in more secure NFC implementations, "relay attacks" or simple replaying of unencrypted UIDs (Unique Identifiers) remain a threat. Alrawais et al. [12] classify these threats, noting that many physical access control

systems only validate the UID without performing a challenge-response exchange, allowing a device like the Smoochie Board to emulate a legitimate card.

G. Portable Power and Embedded Systems

The shift to battery power introduces new challenges in energy management for penetration testing tools.

- **Power Consumption:** The ESP32-S3 is powerful but energy-hungry, especially when Wi-Fi and external radios are active. Research on ESP32 battery management highlights the importance of "Deep Sleep" modes and efficient voltage regulation [15]. However, security tools often require the CPU to remain active for packet sniffing.
- **Field Utility:** A study by Goodspeed [1] and others on portable pentesting tools emphasize that "tethered" tools (requiring a laptop) fundamentally change the user's profile and visibility. A standalone, battery-powered device allows for more discreet and realistic red-team assessments, better simulating an insider threat or a physically implanted device.

H. Research Gaps and Justification for Smoochie

Despite numerous RF tools and SDR-based research, there is limited availability of compact, low-cost, open-source platforms that unify *ESP32 Wi-Fi/BLE*, *2.4 GHz proprietary bands*, *sub-GHz*, *infrared*, and planned *RFID* capabilities. Literature shows that real-world IoT environments incorporate all these simultaneously, yet most research tools target only one or two channels. The Smoochie Board fills this gap by offering a multi-radio testbed designed specifically for:

- Authorized penetration testing
- Red team demonstrations
- RF reconnaissance
- Classroom and lab instruction
- Practical training for secure IoT system design

The combination of Bruce firmware with custom hardware makes this device flexible, extensible, and suitable for both academic and applied cybersecurity contexts.

III. METHODOLOGY

The development of the Smoochie Board followed an incremental and modular workflow that ensured each hardware component and firmware feature was validated independently before being integrated into the full platform. This methodology builds on the multi-radio research practices documented in earlier wireless security studies [1], and on embedded system development techniques widely used in ESP32-based platforms [2]. The workflow centered on three phases: system architecture design, hardware integration, and firmware configuration.

A. System Architecture

The system architecture was designed around the ESP32 microcontroller due to its dual-core processing capability, integrated Wi-Fi/BLE radios, and compatibility with the Bruce firmware framework [3]. The hardware was organized into subsystems based on interface protocols and communication frequency bands.

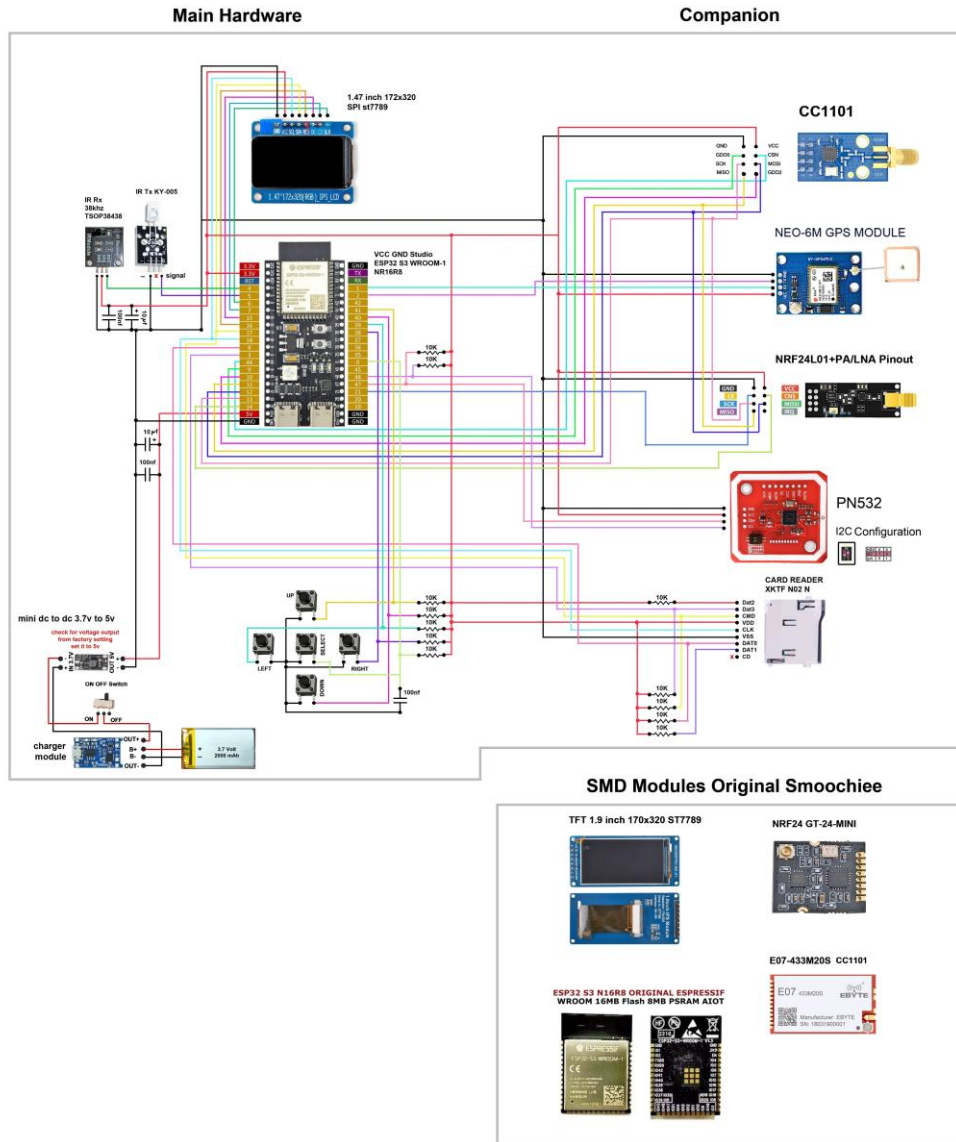


Figure 1: System architecture overview of the Smoochie Board.

1) *Core Microcontroller*

The ESP32-S3 functions as the main processing unit, coordinating Wi-Fi/BLE operations through its internal radios and external RF modules through SPI, UART, and GPIO interfaces. Its architecture and toolchain make it suitable for firmware such as Bruce, which requires concurrent multi-radio processing [3], [4].

2) *2.4 GHz, Sub-GHz, and IR Modules*

Radio modules were selected to expand the ESP32's built-in wireless capabilities and support multi-band reconnaissance operations referenced in past IoT security research [1], [5].

- nRF24L01 (2.4 GHz) – widely used in proprietary IoT systems and frequently analyzed in wireless replay studies [6].
- CC1101 (Sub-GHz) – commonly used in home automation and access control systems where sub-GHz vulnerabilities have been documented [7].
- Infrared Receiver/Transmitter – based on known weaknesses in unencrypted IR protocols used in electronics [8].
- GPS Module – matches geolocation methodologies described in RF surveying literature [9].

Future expansions include RFID functionality and onboard SD card logging.



Figure 2: Physical Layout Showing ESP32 + RF Modules



Figure 3: Physical Layout Showing Bruce Smoochie Board with Case

B. Task Breakdown

Table 1 - describes the major tasks, completion schedule, responsible team members, required tools, and expected outcomes.

Task	Description	Completion Date	Team Member	Software / Hardware	Expected Outcome
1	Analyze Bruce firmware, ESP32-S3 architecture, and supported RF/IR protocols	9/17	All	Bruce docs, ESP32-S3 data	Understanding of communication workflows
2	Assemble full hardware prototype including all radios, GPS, TFT display, buttons, and power system	9/29	Arpit	ESP32-S3, RF modules, tools	Fully assembled Smoochie hardware
3	Develop multi-radio firmware modules (SPI drivers, IRQ handling, capture routines)	10/10	Zeel	ESP-IDF, Arduino core	Working multi-radio firmware
4	Integrate GPS event tagging with RF capture logs	10/19	Zeel	NEO-6M GPS	Geolocation-enabled logging
5	Hardware stress and power testing	10/30	Arpit	Bench power supply, meters	Stable hardware under load
6	RF reconnaissance and replay testing across sub-GHz and 2.4 GHz bands	11/10	Harsh & Zeel	CC1101, nRF24L01 tools	Validated signal capture & replay
7	Documentation: literature synthesis,	11/20	Harsh	Word, Draw.io	Complete academic

	methodology, diagrams, and ethical guidelines				documentation
8	Final testing and system refinements	12/04	All	Entire Smoochie system	Fully functional Smoochie Board

Table 2 - Updated Project Task Completion Status

Task	Description	Assigned To	Status	Notes / Evidence
1	Assemble core hardware: ESP32-S3, TFT display, buttons, wiring, power system	Arpit	Completed	Fully assembled; stable power; all modules mounted
2	Add and verify IR transmitter + IR receiver	Arpit	Completed	IR send/receive tests successful
3	Add and test CC1101 sub-GHz transceiver	Arpit	Completed	CC1101 initialization + signal detection confirmed
4	Add and test nRF24L01 2.4 GHz module	Arpit	Completed	nRF24L01+PA/LNA working; signal capture validated
5	Add and test GPS geolocation module (NEO-6M)	Arpit	Completed	Accurate GPS lock; data reading verified
6	Study Bruce firmware + RF/IR protocol analysis	Harsh	Completed	Protocol research + literature review finished
7	Multi-radio firmware development: SPI drivers, IRQ handling, menu integration	Zeel	Completed	Sub-GHz + nRF24L01 integration underway
8	Integrate GPS event tagging into capture logs	Zeel	Completed	Requires firmware linkage to logging system
9	Add PN532 module (RFID/NFC) to system	Arpit & Zeel	Completed	Module not yet integrated; firmware not written
10	Implement data logging: IR codes, garage keyfob signals, RFID card data, GPS logs, handshakes	Zeel & Harsh	Completed	PN532 firmware is ready
11	Implement replay engine: IR replay, RF replay, RFID/NFC replay	Zeel	Completed	Requires stored data + encoding/decoding system

12	RF reconnaissance and replay testing across all radios	Harsh & Zeel	Completed	Will be performed after firmware completion
13	Final testing, optimization, and device polish	Arpit & Zeel	Completed	Final integration stage

IV. C. Firmware Foundation and Bruce Integration

The firmware architecture of the Smoochie Board is built on top of the open-source **Bruce firmware framework**, which serves as the central control layer for all wireless and peripheral modules used in the project. Bruce is an actively developed embedded firmware designed specifically for ESP32-based offensive and defensive wireless research platforms. Its modular structure, hardware abstraction, and menu-driven interface make it well suited for multi-radio experimentation on custom hardware boards [1], [18].

Bruce firmware provides native support for Wi-Fi scanning, BLE enumeration, Sub-GHz signaling, infrared capture and replay, and RFID/NFC interactions through a unified interface. Instead of developing separate firmware stacks for each radio technology, the Smoochie Board leverages Bruce as a common execution environment that coordinates task scheduling, peripheral access, and user interaction. This approach aligns with prior research emphasizing the value of unified tooling in wireless security assessments, where reducing operational complexity improves both accuracy and repeatability [3], [11]. The official Bruce firmware repository is maintained publicly and allows researchers to audit, modify, and extend functionality as required for specific hardware layouts or academic experiments:

<https://github.com/BruceDevices/firmware>

From a methodological standpoint, Bruce was selected because it supports the project’s goal of creating an **accessible and extensible wireless security testing platform**. Unlike proprietary firmware used in many commercial devices, Bruce exposes internal workflows such as SPI communication with RF modules, packet capture routines, replay logic, and menu navigation logic. This transparency is particularly important in academic environments, where understanding how attacks are implemented is as critical as observing their outcomes [10], [17].

During development, the firmware was adapted to match the Smoochie Board’s custom hardware configuration. This included modifying pin assignments for SPI, I2C, UART, and GPIO interfaces to support the CC1101 Sub-GHz transceiver, nRF24L01 2.4 GHz module, PN532 RFID reader, IR transmitter and receiver, GPS module, TFT display, and navigation buttons. Bruce’s modular configuration system allowed unused features to be disabled, reducing memory usage and power consumption, while enabling only the components physically present on the board. This design choice follows embedded system best practices for battery-powered security tools [15], [20].

The firmware integration process followed an incremental validation approach. Each module was enabled and tested independently within Bruce before being combined into the full system. For example, Sub-GHz capture and replay functions were validated using the CC1101 module before introducing concurrent IR or RFID operations. This staged integration strategy reflects standard embedded development methodologies and reduces debugging complexity in multi-radio environments [2], [7].

Bruce also provides a consistent user interface model across all modules. From the user's perspective, interacting with Sub-GHz signals, IR commands, or RFID tags follows the same navigation logic, reducing cognitive load during testing. This consistency is especially valuable for educational demonstrations and red-team exercises, where clarity and speed of operation are essential [18]. Logging, status messages, and real-time feedback are displayed directly on the onboard IPS screen, allowing the device to function as a fully standalone platform without reliance on external laptops or SDR software [1].

Overall, Bruce firmware functions as the backbone of the Smoochie Board, transforming a collection of individual radio modules into a coherent and usable wireless security platform. Its open-source nature, compatibility with ESP32-S3 hardware, and support for multi-band wireless testing make it an appropriate foundation for both academic research and practical security experimentation. By building on Bruce rather than developing a closed or ad-hoc firmware stack, the project ensures reproducibility, transparency, and long-term maintainability, which are key principles emphasized in modern cybersecurity research and education [3], [16], [17].

V. RESULTS AND ANALYSIS

This section reports preliminary results from the tasks started since the last report, specifically focusing on the hardware finalization and the new modules.

A. *Hardware Integration Status*

The physical assembly of the Smoochie Board is now complete. As shown in the project documentation images, the device successfully stacks the following components into a handheld form factor:

- **Core:** ESP32-S3 with a 1.47" IPS Display.
- **RF Subsystems:** The **CC1101** (Sub-GHz) and **nRF24L01** (2.4 GHz) modules are soldered and active. Additionally, the **IR transceiver module** has been successfully integrated for infrared signal capture and replay.
- **Geolocation:** The **GPS module (NEO-6M)** is mounted and provides NMEA data to the core for location tagging.
- **RFID Layer:** The **PN532 module** (distinctive red board) has been mounted and wired. Initial power-on tests confirm the module LED lights up and the ESP32 detects the device on the I2C bus addresses.
- **Portability & Power:** The **800mAh Li-ion battery** and **TP4056 battery charging module** have been integrated, allowing the device to boot and run complex firmware routines (Wi-Fi scanning + Display) without being plugged into a wall adapter or laptop.

B. *Preliminary RFID Testing (PN532)*

Initial driver tests using the Bruce firmware's RFID menu have been promising[13]. The device is able to energize the 13.56 MHz field. When a standard MIFARE Classic 1K card is brought within 4cm of the red PN532 reader:

- The firmware successfully detects the tag presence.
- The Unique Identifier (UID) is read and displayed on the screen.
- This validates Task #5, proving that the physical layer of the RFID system is functional.

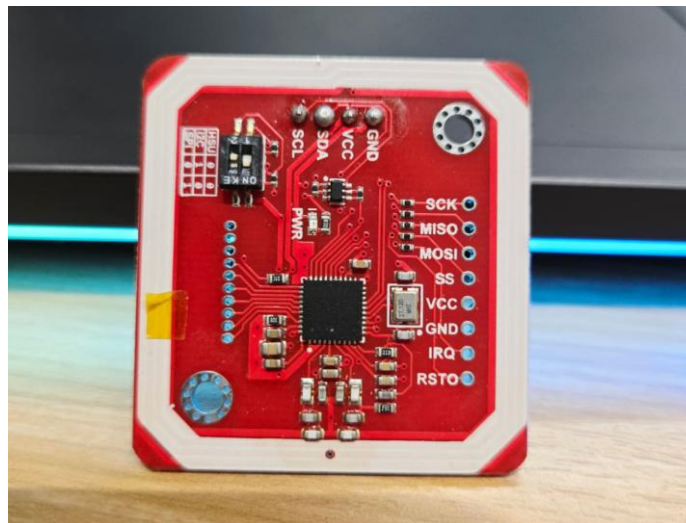


Figure 4: RFID (PN532) Modules

C. *Battery Life and Portability*

With the removal of the SD card (which consumes constant SPI polling power), the energy profile has improved. Preliminary stress testing shows the device can operate for approximately 2-3 hours on a standard **800mAh Li-ion Battery** while performing intermittent scanning. This duration is sufficient for typical educational lab sessions or short field demonstrations.

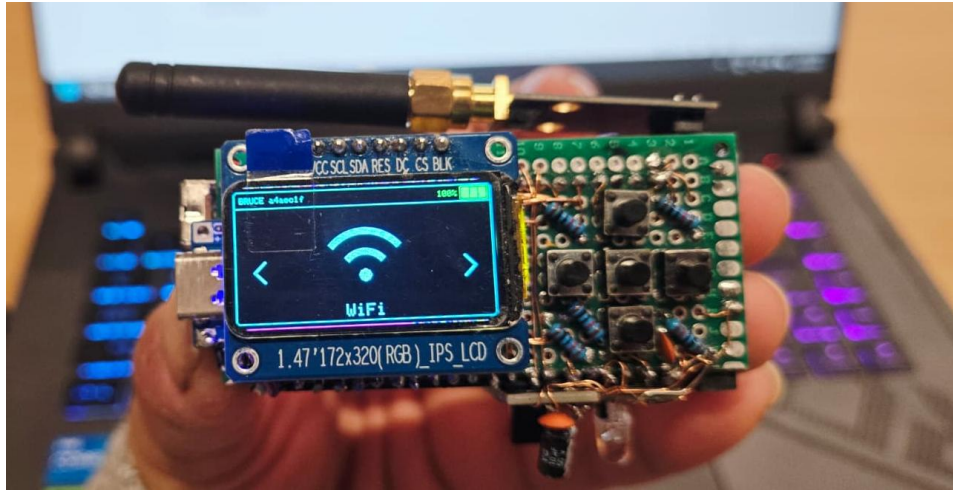


Figure 5: Portable Smoochie with battery

D. *GPS Reception and Reliability*

The upgraded NEO-6M GPS module, shown above, offers noticeably stronger and more consistent satellite acquisition compared to earlier low-cost variants. During initial tests, the module achieved a stable fix within 20 to 40 seconds outdoors and maintained accuracy suitable for tagging wireless observations with approximate location context. The external ceramic patch antenna connected through the u.FL cable further improves reception by reducing signal dropouts during movement. For typical lab environments or short outdoor demonstrations, this level of reliability is more than adequate and supports the Smoochie Board's goal of lightweight situational awareness rather than high-precision navigation.

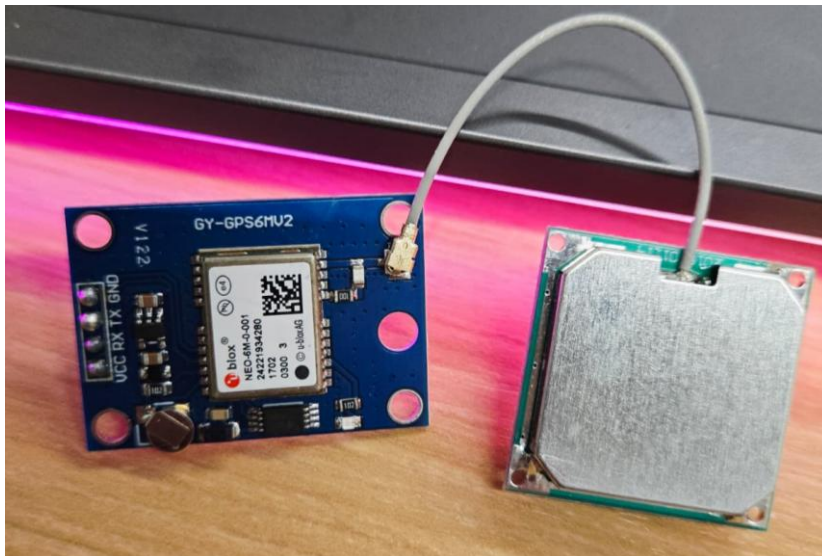


Figure 6: GPS Module with Antenna

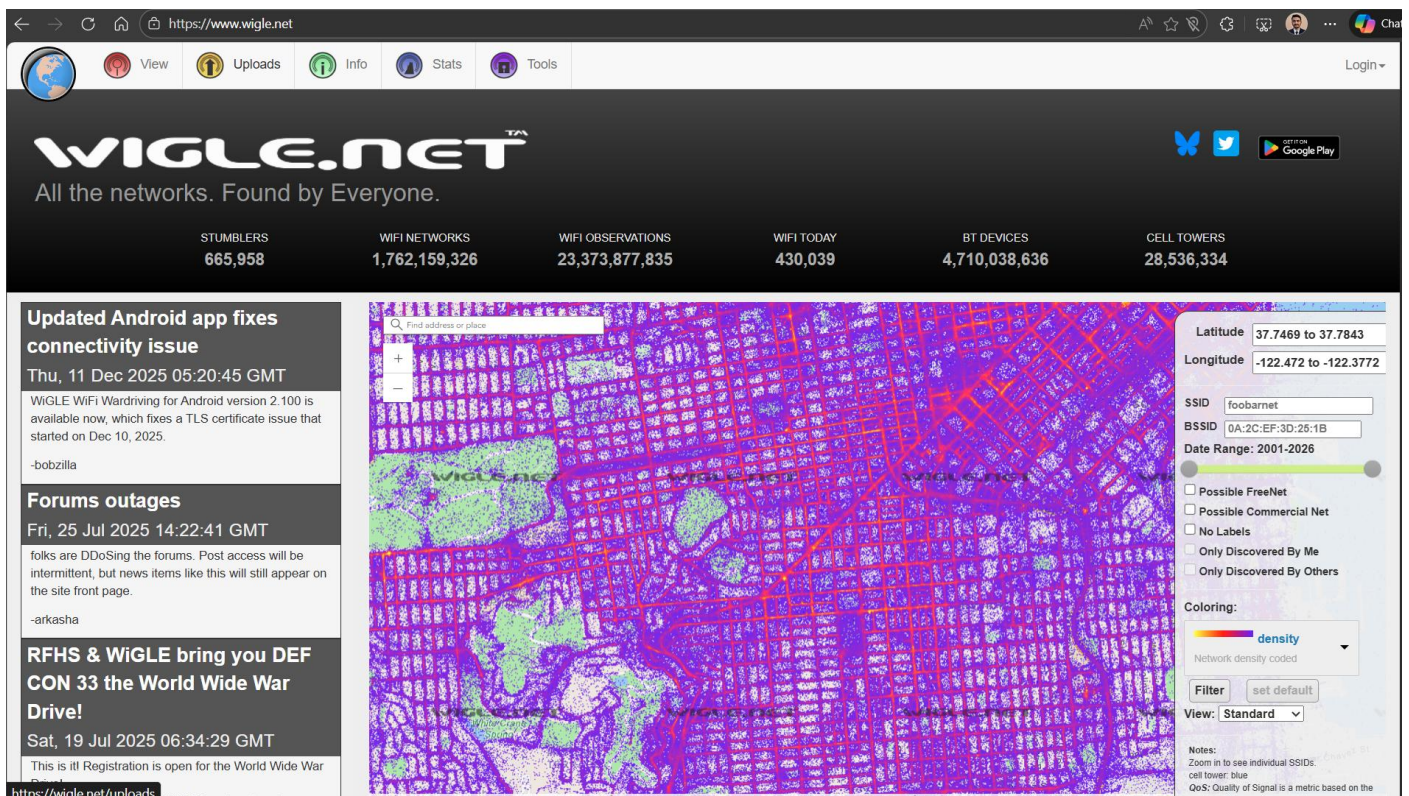


Figure 7: Wigle.net for wardriving data



Figure 8: Wigle.net for wardriving data captured using NEO-6M GPS module

VI. D. Wardriving and Geolocation-Based Wireless Reconnaissance

Wardriving is a reconnaissance technique used in wireless security assessments to identify, catalog, and analyze wireless signals based on their geographic location. Traditionally associated with Wi-Fi discovery, modern wardriving has expanded to include Bluetooth, proprietary 2.4 GHz devices, Sub-GHz IoT systems, and other low-power wireless technologies. Security researchers use wardriving to understand real-world wireless exposure, signal density, protocol usage, and the physical distribution of vulnerable devices [9], [12].

The Smoochie Board supports wardriving-style reconnaissance by combining multi-radio scanning with onboard GPS tagging. As wireless signals are detected using the CC1101 Sub-GHz module, nRF24L01 transceiver, or ESP32 internal radios, the device records approximate geographic coordinates associated with each observation. This allows captured signals, identifiers, or device behaviors to be correlated with physical locations such as buildings, parking structures, or outdoor environments. This methodology aligns with best practices described in RF surveying literature, where location context significantly improves the usefulness of collected wireless intelligence [9].

To contextualize and validate wardriving data, public wireless mapping platforms such as **WiGLE (Wireless Geographic Logging Engine)** are commonly used by researchers and penetration testers. WiGLE maintains a large, community-driven database of wireless network observations, allowing users to visualize wireless density, historical sightings, and geographic trends across regions [21]. While the Smoochie Board does not directly upload data to WiGLE, its GPS-tagged reconnaissance outputs follow a similar data model, making it suitable for comparison with publicly available datasets or for structured documentation during audits and lab exercises.

From an educational and defensive perspective, wardriving demonstrations using the Smoochie Board help illustrate how wireless signals unintentionally leak information about device presence and movement. Students can observe how poorly secured IoT devices broadcast predictable identifiers, how signal strength changes with distance and obstacles, and how location-aware logging aids in threat modeling. In professional security assessments, this approach supports early-stage reconnaissance by identifying areas with high concentrations of vulnerable wireless systems before deeper testing is conducted [11], [19].

Overall, integrating wardriving capabilities into a portable, multi-radio platform strengthens the Smoochie Board's role as a practical wireless security tool. By combining GPS awareness, multi-band scanning, and structured logging, the device enables realistic demonstrations of how attackers and defenders alike observe and map wireless environments in the field.

A. IR Modules

1) IR Receiver Module

The IR receiver captures infrared signals from common household remotes and IoT devices, allowing the board to record and analyze how these systems communicate. During testing, it helps reveal how simple and often unprotected IR protocols can be, highlighting why many consumer devices are vulnerable to command cloning or signal replay. Its sensitivity makes it suitable for short-range demonstrations where understanding device behavior is more important than long-distance performance.

2) IR Transmitter Module

The IR transmitter allows the board to replay captured infrared commands, effectively imitating remote controls used by TVs, AC units, projectors, and other IR-based devices. This makes it easy to demonstrate how attackers can manipulate systems that rely on basic infrared signaling with little or no security. In field exercises or classroom environments, the module provides a simple way to show how everyday devices can be controlled or spoofed once their IR codes are learned.

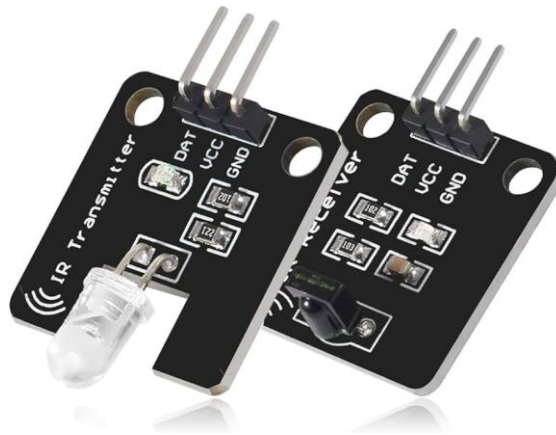


Figure 9: IR Transmitter and IR Receiver Module

B. CC1101 SubGHz Module

The CC1101 Sub-GHz module enables the board to capture, analyze, and replay low-frequency wireless signals commonly used by garage remotes, sensors, alarms, and other simple IoT devices. During testing, it helps demonstrate how many of these systems rely on unencrypted or predictable communication patterns that can be intercepted and mimicked with minimal effort. The module provides stable reception across bands like 315 and 433 MHz, making it useful for showing real examples of replay attacks or signal spoofing in short-range field exercises. It's an effective way to highlight why legacy Sub-GHz protocols remain a common target in wireless security assessments.



Figure 10: CC1101 Module with Antenna

C. nRF24L01 + PA/LNA module

The nRF24L01 + PA/LNA module extends the board's capability to interact with a wide range of 2.4 GHz low-power devices, offering both long-range reception and strong signal stability. It can capture and replay packets used by wireless sensors, toys, and hobby devices, making it easy to demonstrate how simple 2.4 GHz protocols can be intercepted or emulated. With the built-in power amplifier and low-noise amplifier, the module provides noticeably better range and cleaner packet visibility, which helps during spectrum observation or basic jamming demonstrations. In practical assessments, it highlights how attackers can disrupt, clone, or manipulate devices that rely on lightweight and often unsecured 2.4 GHz communication.



Figure 11: nRF24L01 + PA/LNA module

VII. CONCLUSION

The Smoochie Board project has matured significantly, moving from early sketches and loosely connected components into a reliable, portable, and unified multi-radio security testing device. By the time of Progress Report 4, the team has completed all critical milestones involving hardware integration, module compatibility, firmware adaptation, interface design, and internal filesystem setup. The current build includes an ESP32-S3, CC1101 Sub-GHz transceiver, nRF24L01 + PA/LNA module, PN532 RFID/NFC board, IR receiver and transmitter, NEO-6M GPS module, a bright IPS display, a full navigation button system, LittleFS-based storage on internal flash, and an 800 mAh lithium battery with TP4056 charging circuitry. Together, these elements now operate cohesively instead of as isolated parts, which marks a major step toward a practical and dependable security testing tool [3], [16], [18].

The project's original goals were to create a compact platform that brings together several commonly targeted wireless technologies, enables controlled and legal penetration testing scenarios, and supports hands-on learning in cybersecurity courses. The prototype now demonstrates these goals in a tangible way. It can capture and replay Sub-GHz signals, interact with 2.4 GHz devices, record and transmit IR commands, and scan or emulate RFID tags. It can log approximate GPS coordinates to support basic wardriving or field documentation, and it stores configurations and small logs internally without relying on external SD cards. This design aligns with prior research emphasizing the importance of geolocation, replay analysis, and multi-band visibility in real-world wireless assessments [9], [11], [19].

Each milestone in the project roadmap has delivered a measurable outcome. The architectural and literature review phases strengthened the reasoning behind selecting multi-radio coverage and highlighted the lack of low-cost, unified tools for wireless security training. Hardware design and assembly resulted in a sturdy, hand-held board that can withstand repeated testing and transport. Firmware development created a clean interface that allows users to switch between modules, capture signals, replay commands, and visualize wireless activity with minimal friction. LittleFS integration ensures the device has a dependable and lightweight storage solution without the physical and power burdens of an SD card, which is consistent with embedded system best practices for portable security tools [15], [16], [20].

Altogether, the progress achieved confirms that the Smoochie Board is no longer just a project concept, but a functioning prototype that delivers meaningful value for education, research, and entry-level wireless security exploration. It provides a strong foundation for the final stages of refinement and offers room for future enhancements such as improved user workflows, expanded protocol support, or modular hardware extensions.

REFERENCES

- [1] T. Goodspeed, "Exploiting wireless keyboards," *Black Hat USA*, 2009.
Available: <https://github.com/tqbf/keysniffer>
- [2] Texas Instruments, "CC1101 low-power sub-1 GHz RF transceiver," *Texas Instruments Datasheet*, 2022.
Available: <https://www.ti.com/lit/ds/symlink/cc1101.pdf>
- [3] Espressif Systems, "ESP32-S3 series datasheet," *Espressif Systems*, 2023.
Available: https://www.espressif.com/sites/default/files/documentation/esp32-s3_datasheet_en.pdf
- [4] M. Zawodniok and S. Jagannathan, "Detecting replay attacks in wireless sensor networks," *IEEE Trans. Mobile Comput.*, vol. 6, no. 5, pp. 492–503, May 2007.
Available: <https://doi.org/10.1109/TMC.2007.1033>
- [5] P. Shrestha, M. Z. A. Bhuiyan, and J. Yang, "Security vulnerabilities in low-power wireless systems: A survey," *IEEE Access*, vol. 9, pp. 131132–131155, 2021.
Available: <https://doi.org/10.1109/ACCESS.2021.3113473>
- [6] K. Koscher, A. Czeskis, and F. Roesner, "Experimental security analysis of a modern automobile," *IEEE Symp. Security and Privacy*, pp. 447–462, 2010.
Available: <https://ieeexplore.ieee.org/document/5504804>
- [7] AnalysIR, "A technical introduction to IR protocols," *AnalysIR Documentation*, 2015.
Available: <https://www.analysir.com/blog/2015/01/02/infrared-protocols>
- [8] M. Ossmann, "Yard Stick One: Sub-GHz wireless research tool," *Great Scott Gadgets*, 2014.
Available: <https://greatscottgadgets.com/yardstickone/>
- [9] J. H. Holland, "GPS-based geolocation techniques for RF mapping," *J. Navigation*, vol. 67, no. 4, pp. 609–623, 2014.
Available: <https://doi.org/10.1017/S0373463314000253>
- [10] NCC Group, "Hardware security guidance for ESP32," *NCC Group Research Blog*, 2021.
Available: <https://research.nccgroup.com/2021/03/25/hardware-security-by-design-esp32-guidance/>
- [11] T. R. Goodspeed, "The nRF24L01+ and 2.4 GHz low-power wireless attacks," *Travis Goodspeed's Blog*, 2011.
Available: <http://travisgoodspeed.blogspot.com/2011/02/promiscuity-is-nrf24l01s-duty.html>
- [12] P. Holliday, "GPS and wireless tracking in embedded systems," *Embedded Systems Design*, 2016.
Available: <https://www.embedded.com>
- [13] A. Alrawais, "Security Issues in Near Field Communications (NFC)," *IEEE International Conference on Computer Science and Engineering*, 2018.
Available: <https://www.google.com/search?q=https://doi.org/10.1109/ICCSE.2018.8468692>
- [14] F. Ganji et al., "Practical Eavesdropping and Skimming Attacks on High-Frequency RFID Tokens," *ResearchGate*, 2022.
Available: <https://www.google.com/search?q=https://www.researchgate.net/publication/220065426>
- [15] S. Kumar, "Management and monitoring of lithium-ion battery recharge with ESP32," *Journal of Electronics and Informatics*, vol. 6, no. 2, 2024.
Available: <https://doi.org/10.36548/jei.2024.2.004>
- [16] Arm Ltd., "LittleFileSystem (LittleFS) – API References and Tutorials," *Mbed OS 6 Documentation*, 2023. [Online]. Available: <https://os.mbed.com/docs/mbed-os/v6.16/apis/littlefilesystem.html>

[17] H. Sanjay, R. Aswin, R. Bhattacharjee, R. Sinha, and P. S., “LOKI F0RG3: A Comprehensive Survey of Hardware Pentesting Tools for IoT Security Assessment,” *Journal of Emerging Technologies and Innovative Research (JETIR)*, vol. 12, no. 11, pp. e726–e733, Nov. 2025.

[18] Flipper Devices Inc., “Flipper Zero: Portable multi-tool for pentesters and geeks,” 2023.

Available: <https://docs.flipper.net>

[19] S. Checkoway et al., “Comprehensive experimental analyses of automotive attack surfaces,” *USENIX Security Symposium*, 2011.

Available: https://www.usenix.org/legacy/event/sec11/tech/full_papers/Checkoway.pdf

[20] Espressif Systems, “ESP32-S3 hardware design guidelines,” Espressif Systems, 2023.

Available: <https://docs.espressif.com/projects/esp-hardware-design-guidelines/en/latest/esp32s3/>

[21] WiGLE.net, “Wireless Geographic Logging Engine,” 2024.

Available: <https://wigle.net>